Assessment Short Text

Apart from the answers for Task 3, all the others can be found in the project notebook.
In the README.md of the project, I explained how I relied on AI for tasks 1 and 2.

Task 1 : Data Cleaning, Geospatial Wrankling, and Exploitation

1.1 Import and merging

Short description and Peculiarities:

The map (**1.1.png**) shows administrative boundaries for Benin (blue) and Togo (red). Thin lines represent second-level divisions, while thicker lines represent first-level divisions, which makes both levels visible at the same time.
A notable peculiarity is the much denser subdivision in southern Benin, where many small units are clustered near the coast.

1.2 Maize yield

A look at the CSV (fews_net_yield_TGO_BEN.csv) file and the graph **1.2.png** reveals data on maize production—broken down by country, as well as by department and municipality-from 1990 to 2021.
The graph shows production levels in each country, fluctuating between 1 and 1.4 MT/ha depending on the year. In 2011, Benin experienced a peak of around 1.47 MT/ha, followed by a drop of approximately 0.2 MT/ha the following year. A similar pattern occurred in Togo between 2015 and 2016.

1.3 MODIS VI Data

Analysis of graph 1.3.png: Based on visual inspection alone, there is some useful spatial signal, but not enough to claim that vegetation fully explains maize yield.

Across polygons, yes, there is clear variation: some areas are consistently greener than others. Within polygons, not really: each polygon is mapped with a single aggregated value, so internal heterogeneity is hidden. That means we cannot judge field-level or sub-district variability from this map alone.

And based on the same graph but with level 1 feature 1.3(level1).png:

The answer does change between Level 1 and Level 2. At Level 2, you see more local contrast, so it is better for spotting potential links with yield differences.

At Level 1, aggregation smooths those differences, which reduces noise but can also mask meaningful local patterns. So Level 1 is better for broad regional interpretation, while Level 2 is better for exploring yield-related spatial variation.
In short: promising visual evidence, but not sufficient by itself without a quantitative test.

1.4 Reshaping extracted data into a wide format

I reshaped the extracted vegetation-index table into a feature matrix with one row per unique country, admin level 1, admin level 2, and year. For each row, I generated 72 monthly VI features: **2 indices: NDVI, EVI, 3 statistics: mean, min, max, 12 months: 01 to 12**

Feature names follow a consistent naming scheme such as ndvi_mean_01, ndvi_max_09, evi_min_12.

I then merged these vegetation features with the maize yield table using country, admin_1, admin_2, and year. During merging, I handled three common inconsistencies:

- Name formatting mismatch: Administrative names were not always written the same way across sources. I normalized text (lowercase, trimmed spaces, accent removal) and standardized country labels before joining.
- Duplicate yield rows for the same location-year : Some yield records shared the same merge key due to seasonal records. I aggregated them to a single value per key using the mean yield.
- Uneven temporal coverage: MODIS-based VI data do not fully overlap all yield years and may miss some month-location combinations. I preserved all 72 columns and left missing values as NaN rather than imputing unsupported values.

As a result, the merged table is analysis-ready and transparent about data gaps.

Task 3: Modeling and Results interpretation

1. Comparing yields distribution to his log transformation

Based on the 2.1.png plot, I decided to model with log(yield) rather than raw yield because it reduces right-skewness and improves variance stability, which generally leads to better-behaved regression errors and more reliable out-of-sample performance.

2. Model training:

Here are the results:

```
Best alpha: 1.0
Validation R² (CV mean): 0.3408
Test R² : 0.3338
Gap (test − validation): −0.0070
```

Analysis:

The validation (R^2) (0.3408) and test (R^2) (0.3338) are very close, with a small gap of (-0.0070). This means the model's performance is stable from cross-validation to truly unseen data, so there is no strong sign of overfitting.

In practice, the validation score is used during model tuning (to choose hyperparameters like alpha), while the test score is the final unbiased estimate of generalization.

Since both numbers are aligned, we can trust that the model should behave similarly on new data from the same distribution.

At the same time, an (R^2) around 0.33 indicates that the model captures a meaningful part of yield variability, but a large share remains unexplained by the current predictors, which suggests room for improvement with additional agronomic or climate variables.

3. Admin1 aggregation, retrain, and comparison with the previous model

Results:

```
Admin1 best alpha: 10.0
Admin1 validation R2: 0.2718
Admin1 test R2:       0.3658
Admin1 gap:           +0.0941
```

Better test performance: Admin1 aggregate model:

	model	best_alpha	validation_r2	test_r2	gap_test_minus_val
0	Admin2 + one-hot admin2	1.0	0.340811	0.333769	-0.007042
1	Admin1 aggregate + one-hot admin1	10.0	0.271759	0.365817	0.094058

Interpretation and comparison:

The admin1 model has a better test R^2 (0.3658) than the admin2 model (0.3338), but a lower validation R^2 (0.2718 vs 0.3408). So, admin1 looks better on this test split, while cross-validation suggests weaker average performance.

A likely reason is aggregation: at admin1 level, noise is reduced, which can help prediction, but you also lose local detail and some sample size, which can hurt CV results.

Because the admin1 test-validation gap is large (+0.0941), this split may be easier than typical folds, so results should be read with caution.

To improve: use repeated/nested CV, add weather and soil variables, include temporal features, try other models, and use spatially grouped validation.

4. **Publication-Quality plot (2.4.pdf)**

The Scatter plot of predicted vs observed test yields with a 45° reference line. Points close to the line indicate accurate predictions; deviations show prediction error. The model achieves test R^2=0.3617 on n=61 test observations.

Task 3: AI agent architecture proposal

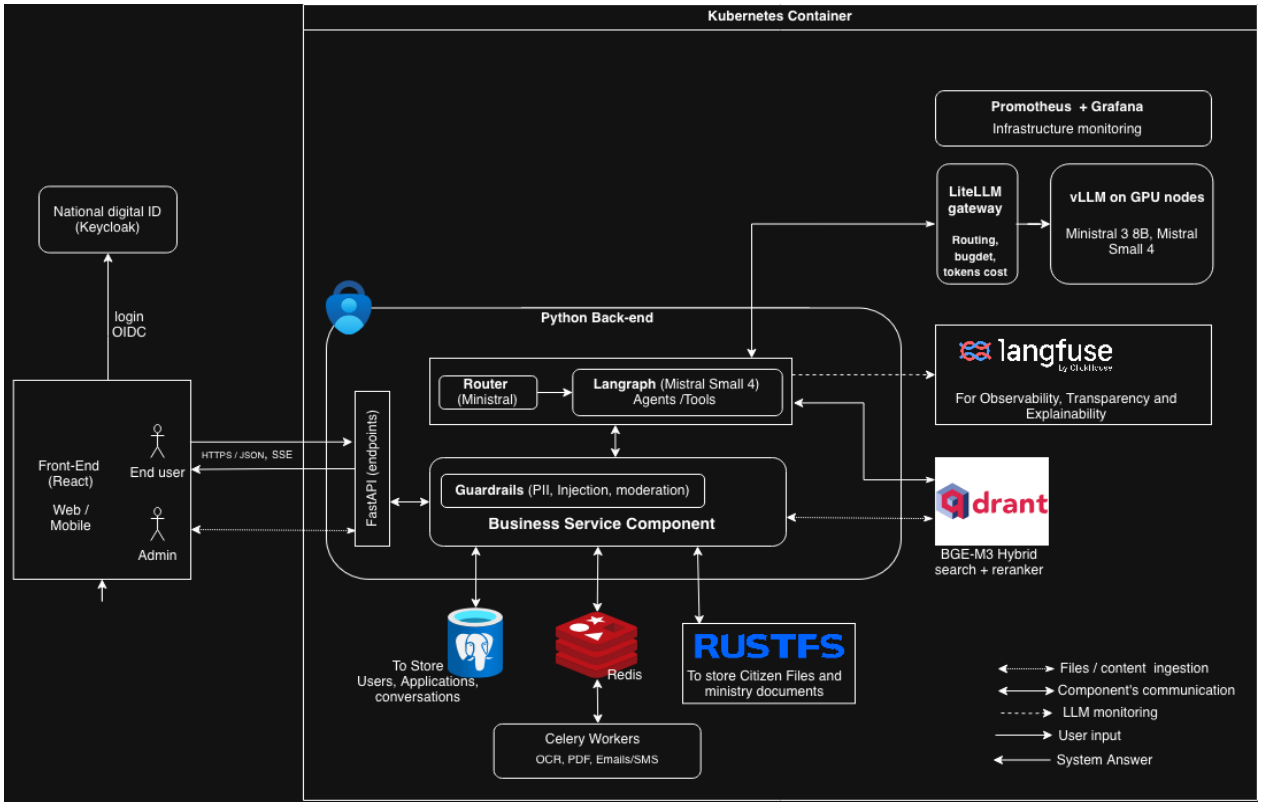
I propose a sovereign, fully open-source conversational assistant that answers citizens' questions about public procedures and lets them file and track requests from the same chat.

The system runs on the government's own Kubernetes cluster, uses open-weight Mistral models (Apache 2.0), and separates two worlds strictly: the AI layer understands, explains and prepares; a deterministic business layer validates, submits and stores.

Quality, latency and cost are measured on every conversation with Langfuse and LiteLLM, and every answer about a procedure must be grounded in an official source.

I used AI for this task understand "How to setup local server with GPU to deploy Open weights LLM and how to expose them to applications". My knowledges on this AI RAG/agent system were focus on cloud (Azure / Bedrock) LLM calls because it's what I did when I worked on such AI applications.

1. Architecture design



The agent calls models only through LiteLLM, which fronts vLLM on the GPU nodes; ministry systems sit outside the cluster and are reached only by the Business Service Component, secured by a scoped token.

Components and their respectives role

Component	Technology (licence)	Role
Front-end	React web + mobile	Chat UI for citizens; back-office view for admins and agents
Identity & access	Keycloak (Apache 2.0), federated with the national digital ID (OIDC)	Authenticates citizens and admins; issues scoped tokens
API layer	FastAPI (MIT)	HTTPS/JSON endpoints, response streaming (SSE), input validation with Pydantic
Guardrails	LangChain middleware, Llama Guard	PII redaction, prompt-injection and moderation checks on input and output
Router	Ministral 3 8B (Apache 2.0)	Classifies intent and service domain; handles simple turns cheaply

Agent	LangGraph (MIT) + Mistral Small 4 (Apache 2.0)	Multi-step reasoning, tool calls, conversation state, human confirmation before any action
Model gateway	LiteLLM	Single OpenAI-compatible endpoint, routing, fallback, per-service keys, budgets, token accounting
Inference	vLLM on GPU nodes	Serves Ministral and Mistral Small 4 (FP8); continuous batching
Business Service Component	Python services	typed tools, business rules, API Calls
Knowledge base	Qdrant (Apache 2.0) + BGE-M3 embeddings (MIT) + bge-reranker-v2-m3	Hybrid dense + sparse search over official procedure sheets
Relational store	PostgreSQL	Users, applications and their status history, conversations, agent checkpoints
Cache & queue	Redis	Sessions, response cache, rate limiting, Celery broker
Background workers	Celery	OCR and document checks, PDF receipts, email/SMS, nightly re-indexing
Object storage	RustFS (Apache 2.0), S3 API	Citizens' uploaded documents, and they are encrypted
Observability	Langfuse (MIT), Prometheus + Grafana	LLM traces, cost, evaluations; infrastructure and GPU metrics

How the bot parses information

The router picks one of three paths:

- **Information**: Hybrid search in Qdrant, answer cites official documents
- **Apply**: Citizen fill the procedure schema propose by the chat and the citizen confirms the summary
- **Track**: Reads PostgreSQL

Only applications confirmed by the citizen are sent to the Business Service Component and ministry systems. If no reliable information is found, the chatbot offers assistance from a human agent.

1. **Normalisation and redaction:** The system cleans the message, detects its language and masks personal information before processing or logging it.

2. **Intent and domain detection:** Ministral identifies the citizen's intent and service domain using a predefined JSON schema. If confidence is low, the chatbot asks for clarification.

3. **Slot filling:** The chatbot collects the information required for each procedure using Pydantic schemas. It validates the provided values and asks for missing details one at a time.

4. **Document processing:** Uploaded files are scanned for viruses and processed using OCR and Ministral vision. Extracted information is checked against the citizen's answers.

5. **Grounded answers:** The chatbot retrieves relevant information from official documents stored in Qdrant. If no reliable answer is found, it refers the citizen to an official source or human agent.

6. **Confirmation and submission:** Before submitting an application, the chatbot shows a summary of the collected information. The request is sent to the Business Service Component only after the citizen confirms it.

2. Evaluation and Monitoring

The chatbot is evaluated through accuracy tests, citizen feedback and production monitoring. Performance targets will be adjusted after the pilot phase.

i. Objective evaluation

Before each release, 50 to 100 questions per service domain are tested with civil servants to check answer accuracy, retrieval quality, tool usage and security.

Metric	Target
Intent and domain accuracy	≥ 95%
Retrieval recall@5	≥ 90%
Faithfulness	≥ 0.90
Answer correctness	≥ 90%
Tool-call accuracy	≥ 95%
Safety	100% of critical cases

ii. User satisfaction

Citizen satisfaction is measured through answer ratings, feedback and task completion rates. The system also tracks abandoned requests, human-agent escalations and applications accepted without corrections. Usability testing is conducted quarterly.

iii. Production monitoring

The system uses Langfuse, Prometheus and Grafana to monitor response times, GPU costs, answer quality, errors and API availability. Alerts help detect performance issues, while unanswered questions are reviewed to identify missing or outdated information.

iv. Hallucination detection

Answers and citations are checked against official sources. Low retrieval scores and negative feedback help identify unreliable responses, which are then reviewed by civil servants.

v. Hallucination prevention

The chatbot relies on official documents and retrieves exact information, such as fees and deadlines, from databases. Uncertain answers are avoided, and confirmed errors are added to future tests. Prompts and models are versioned in Langfuse for easier correction or rollback.

3. Assumptions

The proposed system depends on several technical and operational conditions that may affect the project's cost, scope and timeline.

Technical assumptions

- Digital identity: Citizens authenticate through a national identity system or Keycloak.

- Hosting and AI models: The platform runs on local infrastructure with Kubernetes and sufficient GPU resources across two sites. Mistral Small 4 and Ministral 3 provide the required French-language performance, with data processed locally.

- Language: French is supported initially, with local languages added later.
- Scalability: Server capacity is determined through load testing and adjusted during peak periods.

- Payments: Existing mobile money and card payment services are integrated through APIs.

- Storage: RustFS is used initially, with alternatives such as SeaweedFS if needed.

Operational assumptions

- Content management: Each ministry maintains its procedure information, with updates indexed regularly.
- Human review and support : Civil servants help evaluate chatbot responses and review reported errors. Agents handle requests that the chatbot cannot resolve.

- Data protection: The system follows national privacy laws and informs citizens about AI usage.

- Maintenance: A dedicated technical team manages infrastructure, monitoring and security.

- Deployment: The pilot starts with two priority domains, likely Citizenship and Tax, before expanding to other services.